GNN-for-Airfoil — Interview Deep-Dive

> **Who this is for:** me (the author), preparing to explain and defend this project in ML interviews. **How to read it:** each section starts with a plain-English summary. Technical detail comes after it. Jargon is explained the first time it appears, and there's a Glossary at the end. **Ground rules:** everything is checked against the code in this repo (HEAD `562573c`) and, for the dataset API, against the source of the `airfrans` package (v0.1.5.1, `airfrans/dataset.py`). Anything that can't be checked is marked **UNVERIFIED**.

---

0. TL;DR

- **What:** a graph neural network (GNN) "surrogate" for computational fluid dynamics (CFD). It aims to predict the pressure field around a 2-D airfoil (a wing cross-section) in milliseconds instead of running a slow simulation. Data: the **AirfRANS** dataset (~9.3 GB of RANS simulations). Models: a 2-layer **GCN** baseline, plus **GAT** and **GraphSAGE** variants that also take angle-of-attack and Reynolds number.
- **State:** research prototype. **No results, logs or trained models are committed.** The README lists *expected* outcomes ("Cp MSE < 1e-3, lift error < 10%, R^2 > 0.9"), **not achieved ones**.
- **Biggest things to know before an interview:**
  1. **Data loading doesn't match the real `airfrans` API.** `af.dataset.load()` returns a *tuple* `(list_of_arrays, list_of_names)` (airfrans 0.1.5.1 `dataset.py:56-123`). `data_preprocess.py:185-189` treats it as a list of samples with `.pos` / `.pressure` attributes. On a fresh download, the raw-data path will fail with "no valid samples processed". The only path that can work is loading pre-made `.pth` graph files (`graph_airfrans_data_batch_*`), and **this repo doesn't create them**.
  2. **On that `.pth` path, the train/val/test split is ignored** (`data_preprocess.py:49-53`). All three "splits" are the *first N* graphs of the same list, so **val and test are subsets of train**. Any score from that path measures memorisation, not generalisation.
  3. **Shape bug in the loss.** `create_graph_data` makes `y` shape `(N,1)`. The training loop compares it with `out.squeeze()` of shape `(N,)` (`train_gcn.py:37`, `improved_gnn.py:317`). PyTorch's `MSELoss` then **broadcasts to an N×N matrix**: a wrong loss, and out-of-memory on meshes with ~10^5 nodes.
  4. The target is AirfRANS **kinematic pressure** `p/ρ`, not the pressure *coefficient* Cp. Plots and README call it Cp. "Lift" is computed with a rough approximation that isn't the lift integral.

---

1. One-paragraph pitch (30-second answer)

"Wind-tunnel-quality airflow simulations (RANS CFD) take minutes to hours per design, which is too slow for design optimisation loops. I built a graph-neural-network surrogate on the AirfRANS dataset. Each simulation mesh becomes a graph: nodes carry position, inflow velocity and distance-to-airfoil, and edges come from a k-nearest-neighbour graph. The network predicts the pressure at every node. I

compared a GCN baseline against GAT and GraphSAGE models that also inject the angle of attack and Reynolds number as graph-level conditioning. I evaluated with MSE, R^2 and an integrated lift error. The goal is a surrogate that predicts a whole flow field in milliseconds."

(Be prepared: this pitch describes the design. §8 and §10 explain why there are no trustworthy numbers yet.)

---

2. Plain-language explanation

The problem. When air flows over a wing, it speeds up over the top and slows under the bottom. That creates a pressure difference, and the pressure difference is what makes *lift*. Engineers compute this with **CFD** (computational fluid dynamics): they chop the space around the wing into hundreds of thousands of tiny cells (a *mesh*) and solve the physics equations in every cell. It's accurate but slow.

The idea. If we've already solved about 1,000 of these cases, maybe a neural network can learn the pattern and *instantly* guess the pressure for a new wing shape or angle. That fast stand-in is called a **surrogate model**.

Why a graph? A mesh isn't a neat grid of pixels. Cells are tiny near the wing and huge far away. A **graph** (dots connected by lines) can represent any irregular mesh. Each dot (*node*) is a point in space. Lines (*edges*) connect each point to its neighbours. A **graph neural network** lets each point repeatedly "listen" to its neighbours and update its own guess. That mirrors how pressure at one point depends on the flow around it.

What this repo does.

1. Load the AirfRANS simulations.
2. Turn each one into a graph (each point is linked to its 6 nearest points).
3. Train three kinds of GNN to predict pressure at every point.
4. Measure the error.

The honest caveat. Some parts of the data loading don't match the dataset library, and when data comes from pre-made files, the "test" airfoils were also in the training set. Fixing that is the first step before believing any result.

---

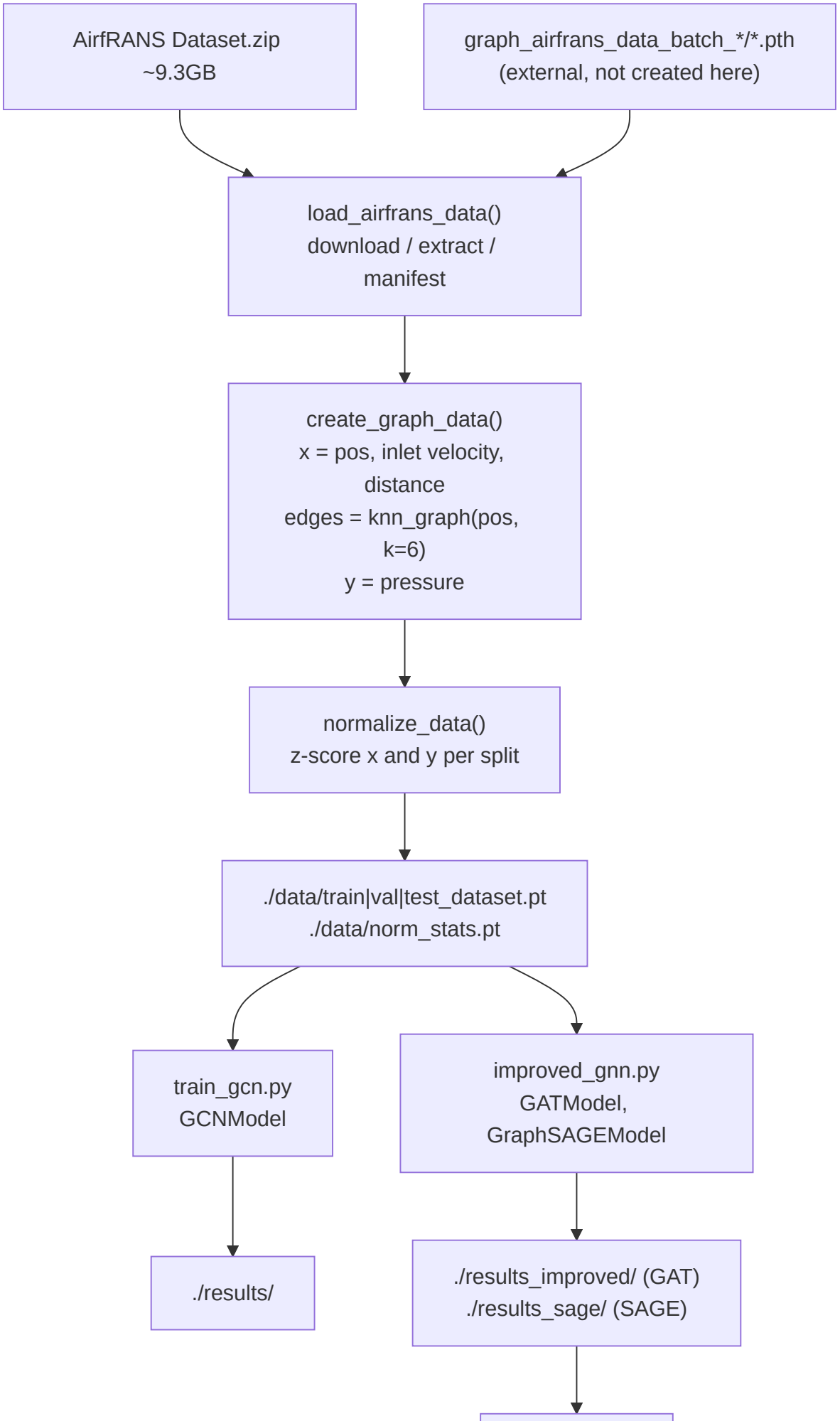
3. Architecture

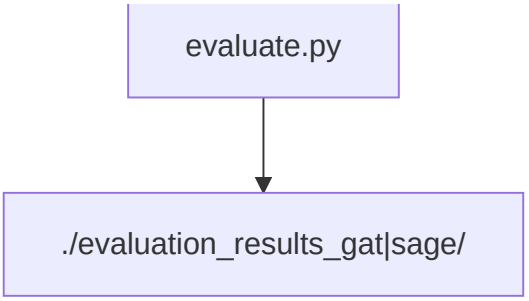
3.1 Components

File	Role
`data_preprocess.py`	Download/extract AirfRANS, convert samples → PyG `Data` graphs (k-NN edges), z-score normalise, save `./data/{split}_dataset.pt` + `norm_stats.pt`, plot first sample
`quick_preprocess.py`	Calls `preprocess_dataset` with 100/50/50 samples
`check_download_status.py`	Reports whether `Dataset.zip` / `manifest.json` / `./data/*.pt` exist
`skip_if_exists.py`	Exit 0 if all four preprocessed files exist

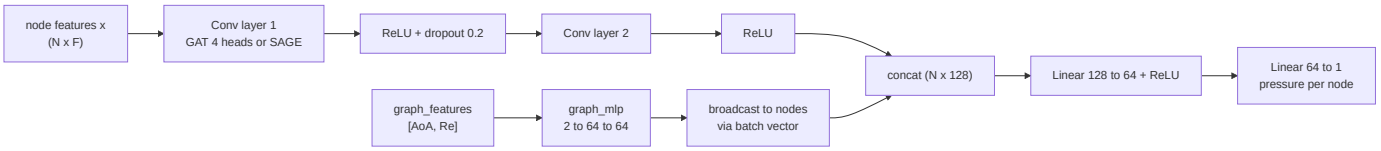
File	Role
`train_gcn.py`	`GCNModel` (2× GCNConv + Linear), training loop, best-on-val checkpoint, test metrics, plots
`improved_gnn.py`	`GATModel`, `GraphSAGEModel` (+ graph-level MLP for AoA/Re), lift-coefficient approximation, training both models
`evaluate.py`	Reload GAT/SAGE checkpoints, per-sample evaluation, chordwise Cp plots

3.2 Architecture diagram

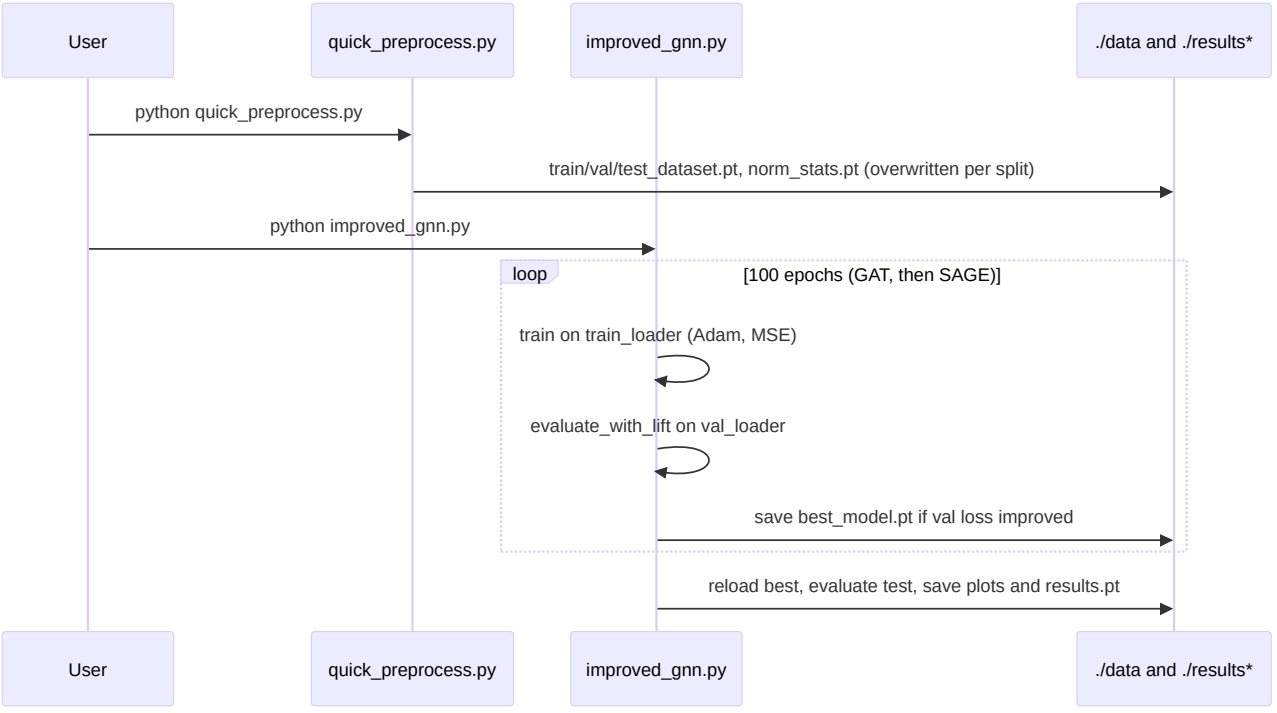

AirfRANS Dataset.zip
~9.3GB
graph_airfrans_data_batch_*/*.pth
(external, not created here)
load_airfrans_data()
download / extract /
manifest
create_graph_data()
x = pos, inlet velocity,
distance
edges = knn_graph(pos,
k=6)
y = pressure
normalize_data()
z-score x and y per split
./data/train|val|test_dataset.pt
./data/norm_stats.pt
train_gcn.py
GCNModel
improved_gnn.py
GATModel,
GraphSAGEModel
./results/
./results_improved/ (GAT)
./results_sage/ (SAGE)



3.3 Model data flow (GAT / GraphSAGE)



3.4 Training lifecycle



---

4. Tech stack

Library	Why chosen	Alternatives & trade-offs
PyTorch (`torch>=2.0`)	The standard deep-learning framework	JAX (fast, less GNN tooling)
PyTorch Geometric (`torch-geometric>=2.3`)	Ready-made `GCNConv`, `GATConv`, `SAGEConv`, `knn_graph`, `Data`/`DataLoader` batching of variable-size graphs	DGL (similar); hand-written sparse message passing (more control, more bugs)
airfrans	Official loader/downloader for the AirfRANS dataset (NeurIPS 2022 Datasets track)	Downloading the raw OpenFOAM files and parsing them with `pyvista`
numpy / scikit-learn	`mean_squared_error`, `r2_score`	`torchmetrics` (GPU-side)

Library	Why chosen	Alternatives & trade-offs
matplotlib	Mesh, Cp-field and chordwise plots	pyvista (proper mesh rendering)
pandas	Listed in `requirements.txt` but **never imported**	—

Why GNNs at all? AirfRANS meshes are unstructured. CNN surrogates (e.g. U-Net on a re-gridded field) need interpolation onto a grid and lose resolution near the airfoil. GNNs and point-cloud networks work on the native nodes.

---

5. Code walkthrough

5.1 `data_preprocess.py`

- **`load_airfrans_data` (`:13-189`)** has two paths:
  1. **Pre-made `.pth` path (`:15-70`)**:
     - If `./airfrans_data/graph_airfrans_data_batch_*/*.pth` exist, load them all (`torch.load(..., weights_only=False)` at `:27` so PyG `Data` objects can be unpickled).
     - If they're already graphs, return `all_data[:n_samples]` (`:49-53`).
     - ⚠️ **`split` is never used here**, so train, val and test all get the *same leading slice* of one list.
  2. **Raw AirfRANS path (`:72-189`)**:
     - Downloads `Dataset.zip` (`af.dataset.download`) or reuses a manually placed zip (validated with `z.testzip()`).
     - Extracts it, searches recursively for `manifest.json`, then calls `af.dataset.load(root, task, train=(split=='train'))`.
     - ⚠️ In airfrans 0.1.5.1 this returns `(data_list, name_list)`. `dataset[:n_samples]` slices **the tuple**, not the simulations.
     - ⚠️ It also loads **every simulation in the split into memory first** (the airfrans loader loops over the whole manifest), which defeats "quick" mode. The README warns about the 9.3 GB download but not about this.
     - ⚠️ `split='val'` and `split='test'` both map to `train=False`, so **val and test are the first 20/50/100 simulations of the same official test set**: identical in quick mode, and overlapping otherwise.
  - Interactive `input()` prompts (`:92`) make it unusable in batch jobs (SLURM/CI).
- **`create_graph_data` (`:191-262`)**:
  - Expects an object with `.pos` / `.inlet_velocity` / `.distance_to_airfoil` / `.pressure` / `.angle_of_attack` / `.reynolds_number`, or a dict with those keys.
    - AirfRANS gives neither. It gives a `(N, 12)` numpy array with columns: `[x, y, u_in_x, u_in_y, sdf, n_x, n_y, u, v, p/ρ, ν_t, is_surface]` (airfrans `dataset.py:105-118`).
    - The matching `Simulation` class uses `.position` and `.sdf`, not `.pos` / `.distance_to_airfoil`.
  - Node features: `x = [pos, inlet_velocity, distance]` (`:216-232`). Surface **normals** and the **surface flag** are not used.
  - **Edges:** `knn_graph(pos, k=6, loop=False)` (`:235`). This ignores the real CFD mesh connectivity, and k-NN in raw coordinates can **connect nodes across the thin trailing edge**, i.e. through the solid airfoil.

    - **Target:** `y = pressure` reshaped to `(N,1)` (`:238-243`). If pressure is missing, it **silently becomes zeros** (`:244-246`).
    - **Graph-level features:** `[angle_of_attack, reynolds_number]` as a 1-D tensor of length 2 (`:249-252`). The defaults (`0.0`, `1e6`) are used silently if missing, so every sample can end up with the same "AoA".
- **`normalize_data` (`:264-297`)**: per-feature mean/std of `x`, and a scalar mean/std of `y`.
    - ⚠️ Called **separately for each split** (`preprocess_dataset:383`). Val and test are normalised with *their own* statistics, which (a) leaks test-set information and (b) puts the splits on different scales.
    - ⚠️ `norm_stats.pt` is saved on *every* call (`:394`), so the file left on disk holds the **test split's stats**. Training code later uses it to denormalise *validation* predictions too.
- `visualize_mesh` (`:299-325`) draws every edge with `ax.plot` in a Python loop. With ~10^5 nodes × 6 edges that's ~10^6 matplotlib calls, which is very slow.

5.2 `train_gcn.py`

- **`GCNModel` (`:11-27`)**: `GCNConv(F→64) → ReLU → dropout(0.2) → GCNConv(64→64) → ReLU → Linear(64→1)`.
- **`train_epoch` (`:29-41`)**: `loss = criterion(out.squeeze(), batch.y)` (`:37`).
    - ⚠️ `out.squeeze()` is `(N,)`, `batch.y` is `(N,1)`, and `nn.MSELoss` broadcasts that to `(N,N)` with only a warning. The loss becomes the mean over all *pairs* of nodes, which is mathematically wrong, and with N ≈ 8 graphs × ~10^5 nodes the allocation is impossible.
    - The fix: `criterion(out, batch.y)` or `out.view(-1)` vs `batch.y.view(-1)`.
- **`evaluate` (`:43-66`)**: MSE and R^2 computed with sklearn on **all nodes of all graphs pooled**, in **normalised** units.
- **`train_model` (`:110-178`)**: Adam (lr 1e-3), 100 epochs, batch size 8. It checkpoints the best validation loss to `best_model.pt`, reloads it and evaluates on test.
    - Standard pattern, but since val ⊆ test (raw path) or val ⊆ train (`.pth` path), "best on val" is not a clean model-selection signal.
- `torch.load('./data/*.pt')` (`:183-185`) has no `weights_only=False`. PyTorch ≥ 2.6 defaults to `weights_only=True`, so loading pickled PyG `Data` lists fails on recent PyTorch.
- `from torch_geometric.data import DataLoader` (`:5`) is the deprecated import location. The current one is `torch_geometric.loader.DataLoader`.

5.3 `improved_gnn.py`

- **`GATModel` (`:11-55`)**:
    - `GATConv(F→64, heads=4)` gives 256 dims, then `GATConv(256→64, heads=1)`.
    - A 2-layer `graph_mlp` embeds `[AoA, Re]` into 64 dims. The result is broadcast to each node of its graph, concatenated, and passed through `combine` (128→64) + ReLU → `Linear(64→1)`.
- **`GraphSAGEModel` (`:57-100`)**: the same, with `SAGEConv` (mean aggregation by default).
- **Broadcasting loop (`:43-47`, `:88-92`)**: a Python loop over graphs in the batch writes `graph_emb[b]` into masked rows.
    - The vectorised form is `graph_emb[batch]`.
    - ⚠️ **Shape issue:** each graph stores `graph_features` as a 1-D tensor of length 2. PyG's batching concatenates 1-D attributes along dim 0, so a batch of B graphs gives shape `(2B,)`, **not `(B,2)`**. `nn.Linear(2, …)` then fails for B > 1. For B = 1 it "works" but `graph_emb[0]` is a single scalar.
    - Store it as shape `(1,2)` per graph to get `(B,2)` after batching.

    - ⚠️ `Re ≈ 10⁶` goes into the MLP **un-normalised**, while AoA is about 0.1 rad. The MLP's first layer saturates on Re.
- **`compute_lift_coefficient` (`:102-130`)**:
    - Picks "surface" nodes as `|y| < 0.01`. That's a horizontal band around the chord line, which includes **wake and field nodes**, not the airfoil surface (AirfRANS has an explicit surface flag).
    - Then `cl = -trapz(cp, x)` over **mixed upper and lower** points sorted by x.
    - The real formula is $C_L = \oint C_p \, (\mathbf{n} \cdot \hat{\mathbf{e}}_L) \, ds/c$, approximately $\int_0^1 (C_{p,\text{lower}} - C_{p,\text{upper}}) \, d(x/c)$ for small AoA.
    - It's also applied to `p/ρ`, not Cp.
- **`evaluate_with_lift` (`:132-205`)**:
    - Splits a batch into graphs with `start_idx = i * (N_total // num_graphs)` (`:176-177`). That assumes **every graph has the same number of nodes**, which is false for AirfRANS meshes. Use `batch.ptr` instead.
    - Denormalises with `norm_stats` (the test stats; see above).
    - Lift error = mean |ΔCl| / mean |Cl| × 100.
- **`train_improved_model` (`:272-376`)** is the same loop as the GCN one, but evaluates lift every epoch. It trains GAT to `./results_improved`, then SAGE to `./results_sage`.

5.4 `evaluate.py`

- Rebuilds GAT/SAGE with `hidden_dim=64` and loads the checkpoints. **`evaluate_unseen_airfoils` (`:89-173`)** runs batch size 1 (which sidesteps the per-graph slicing bug), denormalises, computes MSE/R^2/MAE and lift error, and plots chordwise "Cp" for the first 5 samples.
- `plot_cp_vs_chordwise` (`:9-60`) separates upper and lower by the sign of y **over all nodes**, not surface nodes, so the "chordwise Cp" plot contains the whole field.
- "Unseen airfoils" is the right *intent*. The AirfRANS test split does contain unseen geometries. But see §5.1 for how the split gets lost.

5.5 Utilities

- `check_download_status.py` / `skip_if_exists.py`: friendly status checks so you don't re-download 9.3 GB. They're good developer experience, but they duplicate logic.

---

6. Core concepts & theory

6.1 RANS CFD and AirfRANS

- **RANS** (Reynolds-Averaged Navier–Stokes) solves the time-averaged flow equations plus a turbulence model (AirfRANS uses Spalart–Allmaras) for steady, incompressible 2-D flow.
- **AirfRANS** (Bonnet et al., NeurIPS 2022 Datasets & Benchmarks):
    - About 1,000 simulations over NACA 4- and 5-digit airfoils.
    - Reynolds 2–6 million, angle of attack −5° to +15°.
    - Each mesh has ~10^5 nodes.
    - Tasks: `full` (800 train / 200 test), `scarce` (200 train), `reynolds` and `aoa` (extrapolation splits).
- Targets per node: velocity (u, v), kinematic pressure p/ρ, turbulent viscosity ν_t.

6.2 Pressure coefficient and lift

- $C_p = \dfrac{p - p_\infty}{\frac{1}{2}\rho U_\infty^2}$. It's dimensionless. Values near −1 to −3 on the upper surface mean suction. +1 is the stagnation point.
- With AirfRANS kinematic pressure $\tilde{p} = p/\rho$: $C_p = (\tilde{p} - \tilde{p}_\infty)/(\frac{1}{2}U_\infty^2)$.
- **Lift coefficient:** integrate the pressure (and shear) force on the surface and project it perpendicular to the inflow:

$$C_L = \frac{1}{c}\oint_{\text{surface}} \left(-C_p\,\mathbf{n} + \tfrac{2\boldsymbol{\tau}}{\rho U_\infty^2}\right)\cdot \hat{\mathbf{e}}_L \, ds$$

  Thin-airfoil approximation: $C_L \approx \int_0^1 (C_{p,l} - C_{p,u})\, d(x/c)$.

6.3 Graph neural networks (message passing)

General form for node *i* with neighbours $\mathcal{N}(i)$:

$$h_i^{(l+1)} = \phi\Big(h_i^{(l)},\ \square_{j\in\mathcal{N}(i)}\ \psi(h_i^{(l)}, h_j^{(l)}, e_{ij})\Big)$$

Here □ is a permutation-invariant aggregate (sum, mean or max).

- **GCN** (Kipf & Welling 2017): $H^{(l+1)} = \sigma(\hat{D}^{-1/2}\hat{A}\hat{D}^{-1/2}H^{(l)}W^{(l)})$, with $\hat{A} = A + I$. It's a degree-normalised neighbourhood average followed by a linear map, i.e. a low-pass filter on the graph.
- **GraphSAGE** (Hamilton et al. 2017): $h_i' = W_1 h_i + W_2 \cdot \text{mean}_{j\in\mathcal{N}(i)} h_j$. The node's own state and its neighbours' are kept in separate linear maps.
- **GAT** (Veličković et al. 2018): learned attention weights per edge:

$$\alpha_{ij} = \text{softmax}_j\big(\text{LeakyReLU}(a^\top[Wh_i \,\|\, Wh_j])\big), \quad h_i' = \sigma\Big(\sum_j \alpha_{ij} W h_j\Big)$$

  With multiple heads, outputs are concatenated (layer 1 here: 4 heads × 64 = 256).
- **Receptive field:** 2 message-passing layers on a k=6 graph means each node "sees" only about 2 hops, roughly 40 neighbours. Pressure is a **global, elliptic** quantity: a change at the leading edge affects the whole field. Shallow GNNs are known to under-perform here. Literature models (MeshGraphNets, GraphU-Net, Transolver) use 10–15 layers, multi-scale pooling or attention.
- **Over-smoothing:** stacking many GCN layers makes node features converge. Residual connections and normalisation help.

6.4 Conditioning on global parameters

- The **FiLM-like idea** used here: embed the global scalars (AoA, Re) with an MLP and concatenate them to every node.
- AirfRANS already encodes AoA and |U∞| inside the per-node `inlet_velocity` = U∞(cos α, sin α), so the graph-level MLP is partly redundant. If inputs are properly normalised, it can still help.
- Inputs at very different scales (Re ~10^6 vs α ~0.1) must be normalised. Log-scaling Re is common.

6.5 Normalisation and metrics

- **Z-score** $\tilde{y} = (y - \mu)/\sigma$, fit on **train only** and applied to all splits.

- **MSE** $= \frac{1}{N}\sum(y-\hat{y})^2$. **R²** $= 1 - \frac{\sum(y-\hat{y})^2}{\sum(y-\bar{y})^2}$. Pooling every mesh node lets far-field nodes (thousands of near-freestream values) dominate. **Surface** metrics (Cp on the airfoil, C_L, C_D) are what engineers care about. AirfRANS's benchmark reports the Spearman correlation of predicted vs true C_L and C_D.

---

7. Data

- **Source:** AirfRANS `Dataset.zip` (~9.3 GB) via `airfrans.dataset.download`, or a manual download. Not committed (`.gitignore` excludes `airfrans_data/`, `data/`, `*.pt`).
- **Schema:**
  - Raw (per simulation): array `(N, 12)` = 7 inputs + 4 targets + surface flag.
  - Graph (after `create_graph_data`): `x (N, 2+2+1=5)`, `edge_index (2, 6N)`, `y (N,1)`, `pos (N,2)`, `graph_features (2,)`.
- **Preprocessing:** k-NN graph (k=6); z-score x and y.
- **Splits as implemented:**

Path	train	val	test
Raw AirfRANS (if the API mismatch were fixed)	official train[:n]	official **test**[:n_val]	official **test**[:n_test], which **overlaps or equals val**
`.pth` files	all[:100]	all[:50], a **subset of train**	all[:50], **identical to val, subset of train**

- **Leakage:**
  1. Split collapse (above): **critical**.
  2. Per-split normalisation stats, with test stats used for denormalising everything: medium.
  3. Same geometry at several AoA/Re values *within* AirfRANS: the official `full` split handles this. A random split of `.pth` files would not.

---

8. Evaluation & results

- **Metrics implemented:** node-level MSE and R² (normalised units), MAE (`evaluate.py`), and a "lift error %" (§5.3, approximate and physically incorrect).
- **Results: none committed.** `.gitignore` excludes `results/`, `results_*/`, `*.png` and `*.pt`. The README's "expected outcomes" are **targets, not measurements**. So **no metric can be quoted.**
- **Soundness:** no, as implemented. Split collapse, the loss-broadcast bug, per-split normalisation, the lift approximation, and pooled node metrics mean even a successful run wouldn't measure generalisation to unseen airfoils.
- **What a sound evaluation looks like:**
  - the official AirfRANS `full` test split;
  - train-only normalisation;
  - surface-only Cp MSE;
  - C_L and C_D from proper surface integration (normals and surface flag are in the dataset);
  - Spearman ρ for C_L/C_D;

- comparison against a simple baseline (e.g. an MLP on node features without message passing) and the published AirfRANS baselines.

---

9. Design decisions & trade-offs

Decision	Gained	Lost
k-NN graph (k=6) on positions	Works on any point cloud; no mesh file parsing	Ignores true mesh connectivity; can connect across the airfoil; a fixed k ignores the strong density gradient near the wall
Shallow 2-layer GNNs	Fast, low memory on 10^5-node graphs	Tiny receptive field for a global (elliptic) pressure field
Graph-level MLP for AoA/Re	Explicit conditioning	Redundant with inlet velocity; un-normalised Re
Predict only pressure	Simpler single-output head	AirfRANS benchmarks predict u, v, p, v_t jointly (multi-task helps physics consistency)
Z-score per split	Easy	Leakage + inconsistent scales
Many helper scripts for download state	Friendly for a 9.3 GB dataset	Interactive prompts block automation; duplicated logic

---

10. Honest weaknesses

1. **The raw AirfRANS path cannot work as written.** `af.dataset.load` returns `(data_list, name_list)`. `dataset[:n]` slices the tuple, and `create_graph_data` expects attributes or dict keys that numpy arrays don't have (`data_preprocess.py:185-189`, `:196-210`). Every sample errors out, giving `ValueError("no valid samples processed")` (`:379-380`).
2. **The `.pth` path ignores `split`** (`:49-53`): test ⊂ train. Any reported score would be training performance.
3. **Val and test are the same simulations** on the raw path (both `train=False`, both the first n).
4. **MSE broadcast bug** (`train_gcn.py:37,54`; `improved_gnn.py:162,317`): `(N,)` vs `(N,1)` gives an `(N,N)` loss.
5. **Batched `graph_features` shape** `(2B,)` breaks `graph_mlp` for batch size > 1 (`improved_gnn.py:40,85`).
6. **`norm_stats.pt` overwritten by the last split processed** (`data_preprocess.py:394`), and per-split normalisation.
7. **Lift computation is not lift.** It uses a band $|y| < 0.01$ instead of surface nodes, mixes upper and lower surfaces, and works on p/ρ rather than Cp (`improved_gnn.py:102-130`).
8. **Per-graph slicing assumes equal node counts** (`improved_gnn.py:176-177`).
9. **Target mislabelled as Cp** everywhere in plots and the README.
10. **No results, no tests, no seeds** (`torch.manual_seed` never called), **no CLI args, interactive `input()` prompts**.
11. **Dependency drift:** deprecated `torch_geometric.data.DataLoader`, `torch.load` without `weights_only=False` (breaks on PyTorch ≥ 2.6), and `np.trapz` (deprecated in NumPy 2.0 in favour of `np.trapezoid`).
12. **README provenance:** the quick-start clones `anaya-rs/GNN-Airfoil-Project`, a different repo, which suggests this is a copy of a team project. Be clear in interviews about what you personally

built.

---

11. How I'd improve it (prioritised)

#	Change	Effort	Impact
1	**Rewrite loading against the real API:** `data_list, names = af.dataset.load(root, 'full', train=True)`; slice `data_list`; build graphs from array columns `[0:2]` pos, `[2:4]` inlet velocity, `[4]` sdf, `[5:7]` normals, `[9]` p/ρ, `[11]` surface	0.5 day	Pipeline runs at all
2	**Honest splits:** official `full` train/test; carve val from *train* by geometry name; train-only normalisation saved once	0.5 day	Metrics mean "unseen airfoil"
3	Fix loss shape (`out.view(-1)` vs `y.view(-1)`), `graph_features` shape `(1,2)`, use `graph_emb[batch]` and `batch.ptr`	1 hour	Correct gradients, batching works
4	**Physics-correct metrics:** Cp from p/ρ; surface-only Cp error; C_L/C_D by integrating `-Cp·n` (plus wall shear if available) over flagged surface nodes; Spearman ρ	1 day	Comparable to the AirfRANS benchmark
5	Use the real mesh edges (or radius graph + surface-aware edges) plus relative-position edge features (Δx, Δy, ‖Δ‖)	1 day	Better inductive bias; no edges through the airfoil
6	Deeper encoder-processor-decoder (MeshGraphNets-style, ~10 layers with residuals and LayerNorm) or multi-scale (Graph U-Net); predict u, v, p, v_t jointly	2–3 days	Large accuracy gain on a global field
7	Subsample nodes during training (AirfRANS papers sample ~32k points per mesh), mixed precision	0.5 day	Fits larger batches; faster
8	Reproducibility: seeds, argparse/config, remove `input()`, pin versions, save results JSON + plots, commit a results table	0.5 day	Credible, automatable
9	Baselines: MLP on node features (no edges), plus a comparison against published AirfRANS numbers	0.5 day	Shows the GNN adds value

---

12. How to run it

```
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt          # torch, torch-geometric, airfrans, sklearn, matplotlib, numpy, pandas
python check_download_status.py           # where is the data?
python quick_preprocess.py                # 100/50/50 graphs -> ./data/  (interactive prompts possible)
python train_gcn.py                       # -> ./results/
python improved_gnn.py                    # GAT -> ./results_improved/, SAGE -> ./results_sage/
python evaluate.py                        # -> ./evaluation_results_gat|sage/
```

- **Data:** place `Dataset.zip` (~9.3 GB) in `./airfrans_data/` to skip the download.
- **Expected outcome today:** with the official dataset, preprocessing will likely stop with "no valid samples processed" (§10.1). With external `.pth` graphs it runs, but see the split warning.
- **Env vars:** none.

---

13. Resume bullets (truthful)

- Built a **graph-neural-network CFD surrogate** pipeline in PyTorch Geometric for the **AirfRANS** RANS dataset: unstructured meshes → k-NN graphs, node-wise pressure regression, and **GCN, GAT and GraphSAGE** variants with graph-level conditioning on angle of attack and Reynolds number.
- Implemented data tooling for a **9.3 GB** scientific dataset (resumable download detection, zip validation, manifest discovery) and evaluation utilities for field error, R^2 and an integrated-lift metric.
- *(After fixing §10.1–4)* Audited the pipeline and fixed split leakage, normalisation and loss-shape bugs so results reflect generalisation to unseen airfoil geometries.

> ⚠️ Don't quote "$R^2 > 0.9$" or "lift error < 10%". Those are README *targets*, never measured in this repo.

Glossary

- **CFD**: computational fluid dynamics, numerically solving fluid equations.
- **RANS**: Reynolds-averaged Navier–Stokes, steady CFD with a turbulence model.
- **Surrogate model**: a fast learned approximation of an expensive simulator.
- **Airfoil / chord / angle of attack (AoA)**: wing cross-section; its length; the angle between the chord and the incoming flow.
- **Reynolds number (Re)**: ratio of inertial to viscous forces, $Re = Uc/\nu$.
- **Cp / C_L**: pressure coefficient; lift coefficient.
- **Graph / node / edge**: points and the connections between them.
- **k-NN graph**: connect each point to its k closest points.
- **GCN / GAT / GraphSAGE**: three message-passing GNN layer types (normalised averaging / attention-weighted / separate self-and-neighbour maps).
- **Message passing**: each node updates its state from its neighbours' states.
- **Receptive field**: how far (in hops) information can travel through the network.
- **Z-score normalisation**: subtract the mean, divide by the std.
- **R^2**: fraction of target variance explained by predictions.
- **PyG**: PyTorch Geometric.

GNN-for-Airfoil — Interview Questions & Model Answers

> Companion to `GNN-for-Airfoil.md`. Answers are first person and sized to say out loud in 30–90 s. "Deeper note" = extra detail if probed. 🟢 Easy · 🟡 Medium · 🔴 Hard

If you only prep 10: A1, A3, B2, C1, C3, D1, D3, E1, G1, and all the hostile follow-ups.

(a) Project overview & motivation

A1. 🟢 What is this project?

> It's a graph-neural-network surrogate for airfoil CFD. AirfRANS has about a thousand RANS simulations of 2-D airfoils, each on an unstructured mesh of roughly 10^5 nodes. I turn each mesh into a graph and train GNNs (a GCN baseline, plus GAT and GraphSAGE with angle-of-attack and Reynolds conditioning) to predict pressure at every node. The aim is to replace minutes of CFD with a millisecond forward pass.

A2. 🟢 Why does a CFD surrogate matter?

> Design optimisation and uncertainty quantification need thousands of flow evaluations. If each takes minutes, you can't explore the design space. A surrogate trades some accuracy for orders-of-magnitude speed. You use it to screen designs, then verify the best few with real CFD.

A3. 🟡 What's the current state? Do you have results?

> No trustworthy numbers yet, and I'd rather say that upfront. The README lists target outcomes (R^2 above 0.9, lift error under 10%), but no results are committed. When I audited the code, I found the raw-data loader doesn't match the `airfrans` API, the pre-processed path collapses train/val/test into overlapping slices, and there's a loss-shape bug. So the value today is the design and the audit. The next step is fixing those and running on the official split.

A4. 🟢 Why AirfRANS?

> It's a NeurIPS benchmark dataset with high-fidelity RANS solutions, standard train/test splits including extrapolation tasks in Reynolds and AoA, and published baselines to compare against. It's also big enough (about 9 GB) to be realistic.

A5. 🟡 Why is this relevant to an ML role and not just mechanical engineering?

> It's geometric deep learning on irregular data: variable-size graphs, conditioning on global parameters, and physics-aware evaluation. The same skills apply to molecules, meshes, point clouds and recommender graphs. My mechanical background is what let me spot that the "lift" metric wasn't physically correct.

(b) Architecture & design

B1. 🟢 Walk me through the pipeline.

> Load the simulations, convert each to a PyG `Data` object (node features: position, inlet velocity and distance-to-airfoil; edges: a 6-nearest-neighbour graph; target: pressure), z-score normalise, and save as `.pt`. Then the training scripts train GCN, GAT and SAGE with Adam and MSE, keep the best checkpoint on validation, and evaluate on test with MSE, R^2 and lift error.

B2. 🟡 Why build a k-NN graph instead of using the mesh?

> It's simpler: you only need point positions, not the mesh connectivity files. But it's a compromise. The CFD mesh is graded, very fine near the wall, so a fixed k gives very different spatial reach in different regions. And k-NN in raw coordinates can link nodes across the thin trailing edge, i.e. through the solid. Using the real mesh edges, or adding relative-position edge features, is better.

B3. 🟡 How is angle of attack/Reynolds injected?

> A small MLP embeds `[AoA, Re]` into a 64-d vector, which is broadcast to every node of that graph and concatenated after the convolutions. One caveat: the per-node inlet velocity, $U_\infty(\cos\alpha, \sin\alpha)$, already encodes AoA and speed, so it's partly redundant. And Re (about 10^6) must be normalised or log-scaled first, which the code doesn't do.

B4. 🟡 Why three architectures?

> To compare aggregation schemes. GCN is degree-normalised averaging (a low-pass filter). GraphSAGE keeps the self and neighbour terms separate, which preserves sharper local features. GAT learns per-edge attention, which in principle can focus on the upstream or near-wall neighbours that matter. It's a sensible ablation, as long as the evaluation is valid.

B5. 🔴 What architecture would you use if you started again?

> An encoder-processor-decoder like MeshGraphNets. Encode node and edge features (relative positions) with MLPs, run about 10–15 message-passing blocks with residuals and LayerNorm, then

> decode u, v, p and v_t jointly. Pressure is governed by an elliptic equation, so information has to travel across the whole domain. Either deep message passing, multi-scale pooling (Graph U-Net), or a global attention mechanism (Transolver-style) is needed. Two layers isn't enough.

(c) Code-level / implementation

C1. 🔴 What does `af.dataset.load` return, and how does the code use it?

> In airfrans 0.1.5.1 it returns a tuple: a list of `(N, 12)` numpy arrays and a list of simulation names. The columns are x, y, inlet u, inlet v, signed distance, two normal components, then u, v, p/ρ, v_t and a surface flag. The code does `dataset[:n_samples]`, which slices the *tuple*, then expects each sample to have `.pos` or dict keys. Arrays don't, so every sample throws and preprocessing ends with "no valid samples processed". The fix is unpacking the tuple and indexing columns.

C2. 🟡 What's wrong with the `.pth` loading branch?

> It ignores the `split` argument and returns `all_data[:n]` for train, val and test. So val and test are the first 50 graphs, which are also inside the first 100 training graphs. The model is tested on data it trained on.

C3. 🔴 There's a broadcasting bug in the loss. Explain it.

> `y` is stored as `(N,1)`. The model output is squeezed to `(N,)`. `nn.MSELoss` broadcasts `(N,)` against `(N,1)` to an `(N,N)` matrix and only warns. So the loss is the mean squared difference between every prediction and every target, which is meaningless. With 10^5-node meshes in batches of 8, that's an impossible allocation. The fix is to keep both as `(N,1)` or flatten both.

C4. 🟡 How does PyG batch multiple graphs, and where does this code get it wrong?

> PyG concatenates node tensors along dim 0, offsets `edge_index`, and adds a `batch` vector mapping nodes to graphs. Graph-level attributes stored as 1-D tensors get concatenated too. Two features per graph becomes `(2B,)`, not `(B,2)`. The graph MLP expects `(B,2)`. Storing `graph_features` as `(1,2)` per graph fixes it, and then `graph_emb[batch]` broadcasts to nodes without the Python loop.

C5. 🟡 How is the per-graph lift computed in a batch, and is that right?

> It assumes each graph has `N_total / num_graphs` nodes and slices evenly. AirfRANS meshes have different node counts, so the slices straddle graphs. `batch.ptr` gives the exact boundaries. `evaluate.py` happens to avoid this by using batch size 1.

C6. 🟢 How does checkpointing work?

> After each epoch, if validation loss improves I save `state_dict` to `best_model.pt`. After training I reload the best and evaluate on test. It's early-stopping-style model selection, without stopping early.

C7. 🟡 What will break on a newer environment?

> `torch.load` defaults to `weights_only=True` from PyTorch 2.6, so loading pickled `Data` lists fails without `weights_only=False`. `torch_geometric.data.DataLoader` is the deprecated location (it's `torch_geometric.loader` now). And `np.trapz` is deprecated in NumPy 2. I'd pin versions and update the imports.

C8. 🟡 How are features normalised and what's the issue?

> Z-scores computed over all nodes of a split. The issue is it's done separately per split, and `norm_stats.pt` gets overwritten each time, so the saved stats are the test set's. Normalisation should be fit once on train and applied everywhere.

(d) Underlying ML/CS theory

D1. 🟡 Explain message passing.

> Each layer, every node computes messages from its neighbours' features, aggregates them with a permutation-invariant function (sum, mean or max), and combines the result with its own state through a learned function. After L layers, a node's representation depends on its L-hop neighbourhood. GCN, SAGE and GAT are different choices of message and aggregation.

D2. 🟡 What's the GCN propagation rule?

> H' = σ(D̂^{-1/2} Â D̂^{-1/2} H W), with Â = A + I. So it's a symmetric degree-normalised average of self and neighbours, then a linear map. Spectrally it's a first-order approximation of a graph low-pass filter. That's why deep GCNs over-smooth.

D3. 🔴 Why might a 2-layer GNN fundamentally struggle to predict pressure?

> Pressure in incompressible flow satisfies a Poisson-type equation. It's elliptic, so every point depends on boundary conditions everywhere. A 2-layer GNN on a 6-NN graph has a receptive field of about 40 nearby nodes, a tiny patch of a 10^5-node mesh. The model can't "see" the airfoil from most of the field. You need depth, multi-scale hierarchies, or global attention.

D4. 🟡 GAT vs GraphSAGE: when would each win?

> GAT helps when neighbours have unequal importance and the graph is noisy: attention can down-weight irrelevant ones. SAGE is cheaper, and its separate self/neighbour weights avoid washing out a node's own signal. For dense, regular meshes SAGE-style or edge-conditioned convolutions are often as good. GAT's attention doesn't use geometry unless you add edge features.

D5. 🟡 What's the pressure coefficient, and why normalise by dynamic pressure?

> $C_p = (p - p_\infty) / (\tfrac{1}{2}\rho U_\infty^2)$. Dividing by dynamic pressure makes it dimensionless, so cases at different speeds are comparable, and Cp distributions are what aerodynamicists read. AirfRANS stores p/ρ, so you'd subtract the freestream and divide by $\tfrac{1}{2}U_\infty^2$.

D6. 🔴 How do you compute lift from a pressure field correctly?

> Integrate pressure (and wall shear) over the airfoil surface, using the surface normals, and project onto the direction perpendicular to the freestream: $C_L = (1/c)\oint(-C_p\, n)\cdot e_L\, ds$, plus a viscous term. AirfRANS gives you the surface flag and normals. The thin-airfoil shortcut is $\int(C_{p,\text{lower}} - C_{p,\text{upper}})\, dx/c$. My code instead integrates $-C_p\, dx$ over a horizontal band of points near y=0. That's not the surface, and it mixes the two sides.

D7. 🟢 What's over-smoothing?

> As you stack averaging layers, node representations converge towards each other, so you lose discriminative local information. Residual connections, normalisation, gating or jumping-knowledge connections mitigate it.

(e) Evaluation & metrics

E1. 🔴 Is your evaluation valid? What would make it valid?

> As written, no. Test overlaps train on the `.pth` path and equals val on the raw path. Normalisation is per split. Metrics pool all nodes in normalised units. To make it valid: the official AirfRANS `full` split, validation carved from training by geometry, train-only normalisation, surface Cp error, integrated C_L and C_D with Spearman correlation as in the AirfRANS paper, and a no-message-passing MLP baseline to show the graph structure helps.

E2. 🟡 Why is pooled-node R^2 misleading here?

> Most nodes are far from the airfoil, where pressure is close to freestream. A model that's right there and wrong at the leading-edge suction peak can still get a high R^2. Engineers care about surface pressure and integrated forces, so those should be the headline metrics.

E3. 🟡 What does "unseen airfoil" generalisation require?

> The test set must contain geometries the model never saw at any AoA/Re. The official AirfRANS split guarantees that. A random split over simulations would leak the same shape at different conditions.

E4. 🟡 How would you test extrapolation?

> AirfRANS has `reynolds` and `aoa` tasks, where test conditions lie outside the training range. Surrogates usually degrade badly there. Reporting both interpolation and extrapolation is honest and informative.

E5. 🟢 What baseline would you include?

> A per-node MLP on the same features (no edges) tells you whether message passing adds anything. A k-nearest-simulation interpolation baseline is also useful. And the published AirfRANS results for GraphSAGE, PointNet and the MLP give an external yardstick.

(f) Scaling, production & system design

F1. 🟡 Each mesh has ~10^5 nodes. How do you train efficiently?

> Subsample nodes per step (AirfRANS papers sample around 32k points), use mixed precision, and use neighbour-sampling loaders. At inference you run on the full mesh. Also precompute graphs once and store them compactly instead of re-running k-NN every time.

F2. 🟡 How would you deploy a surrogate in a design loop?

> Wrap the model with a preprocessing step that turns a new geometry into a mesh, or a point cloud, plus features. Return Cp on the surface plus C_L and C_D, with uncertainty, e.g. from an ensemble or MC dropout. Then gate the decision: if uncertainty is high or the inputs are outside the training range, fall back to real CFD.

F3. 🔴 How do you make the surrogate trustworthy for engineers?

> Physics checks: lift-curve slope near 2π/rad at low AoA, a Cp of about 1 at stagnation, smooth Cp along the surface. Out-of-distribution detection on geometry and conditions. Calibrated uncertainty. And continuous comparison against spot-checked CFD runs. Optionally, physics-informed losses (divergence-free velocity) if predicting the full state.

F4. 🟢 Storage concerns?

> The raw dataset is 9.3 GB. Preprocessed graphs with `edge_index` are larger than the raw arrays (2×6N integers per graph). I'd store float16 features and int32 indices, or rebuild edges on the fly.

(g) Failure modes & debugging

G1. 🔴 Preprocessing prints "error processing sample 0", then "no valid samples processed". How do you debug it?

> First, inspect `type(dataset)`. It's a tuple from `af.dataset.load`. Then `type(sample)` inside the loop, which is a list, not a simulation. Then check the library docstring for the return format. That's the root cause: unpack `data_list, names` and index array columns. Lesson: when a broad `except` swallows errors, print the exception type and the sample type before anything else.

G2. 🟡 Training immediately runs out of memory. Why?

> Most likely the `(N,)` vs `(N,1)` MSE broadcast creating an N×N tensor. Otherwise it's batch size 8 on 10^5-node graphs with GAT's 4 heads. Check `out.shape` and `batch.y.shape` first, then reduce batch size or subsample nodes.

G3. 🟡 Validation loss is suspiciously close to training loss from epoch 1. Why?

> On the `.pth` path, validation graphs are a subset of training graphs. Hash the sample IDs or names in each split and assert they're disjoint.

G4. 🟡 The lift-vs-AoA plot looks flat. What would you check?

> Whether AoA is actually varying. `create_graph_data` defaults AoA to 0 if missing, so all samples might share one value. Also, the "lift" integrates a band of points near y=0 rather than the surface, which can hide the true trend.

G5. 🟢 Results differ between runs. Why?

> No seeds set, shuffling, dropout, and non-deterministic scatter operations on GPU. Set `torch.manual_seed`, `numpy` seeds and deterministic flags where possible, and report the mean ± std over several seeds.

(h) "Why didn't you..." / alternatives

H1. 🟡 Why not a CNN on a grid?

> You'd interpolate the unstructured mesh onto a Cartesian grid, losing the fine near-wall resolution where the important physics is, and wasting cells inside the airfoil. Graphs or point clouds keep the native resolution.

H2. 🟡 Why not a physics-informed neural network (PINN)?

> PINNs solve one PDE instance by minimising residuals. They're slow to train per case and don't amortise across geometries. A data-driven surrogate amortises: train once, then predict new cases instantly. Hybrids, data-driven plus physics regularisation, are a good middle ground.

H3. 🟡 Why not neural operators (FNO/DeepONet)?

> FNO assumes regular grids for its FFTs. Geometry-aware variants (Geo-FNO, GINO) exist and would be a strong comparison. GNNs were the natural first step for unstructured meshes.

H4. 🟢 Why not predict velocity and turbulent viscosity too?

> I started with pressure because it determines lift and is a single output. Jointly predicting u, v, p and v_t is the AirfRANS benchmark task, and multi-task learning often regularises the shared representation. It's on the improvement list.

H5. 🟡 Why k=6?

> It's roughly the number of mesh neighbours in a 2-D triangulation. But it wasn't tuned, and with graded meshes a radius-based graph or the true mesh edges would be more principled.

(i) Behavioural (STAR)

I1. 🟡 Tell me about debugging something that "worked" but was wrong.

> **S:** The training scripts produced loss curves and plots, so it looked like it worked. **T:** Before presenting results, I wanted to verify the data path end to end. **A:** I read the `airfrans` source and found `load()` returns a tuple, so the raw path couldn't produce samples, meaning results must have come from pre-processed `.pth` files. Then I saw that path ignores the split argument, so test was inside train. **R:** I stopped treating any numbers as results and wrote down the fixes: the correct API usage, official splits, train-only normalisation and the loss-shape fix. Lesson: assert split disjointness in code.

I2. 🟢 Tell me about working with a large dataset under constraints.

> **S:** AirfRANS is 9.3 GB and can't be partially downloaded. **T:** Make it usable for teammates on slow connections. **A:** I added detection of manually placed zips with integrity checks, recursive manifest discovery, a status script, and a quick-preprocess mode. **R:** People could reuse a downloaded zip instead of re-fetching. In hindsight I'd remove the interactive prompts so it also works in batch jobs.

I3. 🟡 Tell me about bringing domain knowledge to an ML problem.

> **S:** The project reported a "lift error". **T:** Check that the metric meant what it claimed. **A:** From aerodynamics, lift comes from integrating pressure times the surface normal over the airfoil. The code integrated a horizontal band of field points and mixed upper and lower surfaces, using kinematic pressure, not Cp. **R:** I specified the correct computation using AirfRANS's surface flag and normals. That's the kind of check a pure-ML person might miss.

I4. 🟡 Tell me about a collaboration on this project.

> *(Fill in truthfully.)* The README's quick-start points at `anaya-rs/GNN-Airfoil-Project`, so this was a team effort. Template: "I owned X (e.g. the GAT/SAGE conditioning and evaluation). My teammate owned Y. We split work by pipeline stage. In hindsight we needed a shared contract for data shapes, since the `(N,1)` vs `(N,)` mismatch was exactly an interface problem between stages."

Hostile follow-ups

X1. 🔴 "So you've never actually trained this on the real dataset?"

> Not through the raw path in this repo, because it can't load the official format. Any runs went through pre-processed `.pth` graphs, and that path collapses the splits. So I don't claim generalisation results. What I can show is the design, the audit and a concrete fix list. I'd rather tell you that than quote a number I can't defend.

X2. 🔴 "Your 'lift' isn't lift. Why should I trust anything else?"

> Fair challenge. The lift function is a placeholder approximation, and the comment in the code even says "simplified". The rest (MSE and R^2 on node pressure) is mechanically correct but measured on bad splits. The right move is to rebuild evaluation around the AirfRANS surface flag and normals, which I've specified.

X3. 🔴 "Two-layer GNN on 10^5 nodes: did you not know pressure is global?"

> I did by the end. That's exactly the limitation I'd fix first on the modelling side. The shallow model was a baseline to get the pipeline running. The literature (MeshGraphNets, AirfRANS baselines,

> Transolver) uses much deeper or multi-scale models for this reason.

X4. 🔴 "Your MSE is an N×N broadcast. How did training even run?"

> If the `.pth` graphs stored `y` as 1-D, the shapes would match and it would run. On graphs produced by `create_graph_data`, it wouldn't, or it would hit OOM. Either way it's an interface bug between preprocessing and training. A single shape assert would have caught it.

X5. 🔴 "Why is val normalised with different statistics than train?"

> Because normalisation is called per split. It's a bug. It leaks test information and changes the target scale per split, and the saved stats end up being the test set's. Fit on train once, save, apply everywhere.

X6. 🔴 "The README says $R^2 > 0.9$. Where's that from?"

> It's listed under "expected outcomes", a target, not a measurement. I should make that clearer, or remove it until I have a run on the official split.

X7. 🔴 "Did you write this, or is it your teammate's repo?"

> *(Answer truthfully.)* The README clones from a teammate's repository. Be ready to state which files and decisions are yours and to walk through them line by line.

X8. 🟡 "Isn't AoA already in the inlet velocity? Why the graph MLP?"

> Yes. Inlet velocity is $U_\infty(\cos\alpha, \sin\alpha)$, so AoA and speed are encoded per node. The graph-level path mainly adds Re, which isn't otherwise present, but it's un-normalised. I'd keep a global conditioning path but feed it log-Re and normalised AoA, or use FiLM-style modulation of each layer.